

[See Topic 11 → Trie Masterclass](#)

GOAL: Master Graph representations, Graph Construction Workshop, DFS/BFS traversals, Multi-Source BFS, Topological Sort (Kahn's), Disjoint Set Union (DSU), Dijkstra's Shortest Path, and Kruskal's MST!

⚡ AHA MOMENT #1: GRAPHS VS TREES & THE VISITED TRACKER

A **Tree** is simply an undirected acyclic graph with $V - 1$ edges and a single root. A **Graph** can contain arbitrary cycles, multiple disconnected components, and directional edges!

THE GOLDEN GRAPH LAW: Every graph traversal (DFS/BFS) MUST maintain a `visited[]` array or `visited[][]` matrix to prevent infinite loops caused by cycles!

1 CORE GRAPH TERMINOLOGY

- Vertices (V) & Edges (E):** Nodes and links connecting them.
- Directed (Digraph) vs Undirected:** One-way arrows ($u \rightarrow v$) vs two-way links ($u \leftrightarrow v$).
- Weighted vs Unweighted:** Edges carrying cost/distance vs uniform unit cost.
- DAG (Directed Acyclic Graph):** Directed graph with zero cycles (Required for Topological Sort!).

2 TREE VS GRAPH PARADIGM COMPARISON

Feature	Tree	Graph
Root Node	Single Root (Top-Down)	No designated root
Cycles Allowed	❌ Never (Acyclic)	✅ Yes (Cyclic or DAG)
Visited Tracking	Not needed (Parent only)	✅ MANDATORY (<code>visited[]</code>)
Edges Count	Exactly $V - 1$	Arbitrary (0 to V^2)

🔗 VISUALIZING DIRECTED DAG VS UNDIRECTED CYCLIC GRAPH

1. DIRECTED ACYCLIC GRAPH (DAG): $[0] \rightarrow [1] \rightarrow [2] \mid \blacktriangle \text{ ————— } \blacktriangledown$
(Cycle: $0 \leftrightarrow 1 \rightarrow 2 \rightarrow 0$!)

2. UNDIRECTED CYCLIC GRAPH: $[0] \longleftrightarrow [1] \blacktriangle \mid \blacktriangledown \text{ ————— } [2]$

🚩 **KEY TAKEAWAY:** Always identify if the graph is Directed vs Undirected, Weighted vs Unweighted, or a Grid before picking an algorithm!



💡 AHA MOMENT #2: GRAPH CONSTRUCTION IS STEP ZERO

Interviews do not give you a pre-built Graph object. They hand you an **Edge List** `int[][] edges`. Master constructing an **Adjacency List** `List[]` within 5-10 seconds before writing ANY traversal logic!

🔧 STEP-BY-STEP ADJACENCY LIST GROWTH ANIMATION FOR UNDIRECTED EDGES [(0,1), (0,2), (2,3)]

INITIALIZATION: `adj[0] = [], adj[1] = [], adj[2] = [], adj[3] = []` **STEP 1: Process Edge (0, 1)** Add 1 to `adj[0]`, Add 0 to `adj[1]` `adj[0] = [1] adj[1] = [0]` **STEP 2: Process Edge (0, 2)** Add 2 to `adj[0]`, Add 0 to `adj[2]` `adj[0] = [1, 2] adj[1] = [0] adj[2] = [0]` **STEP 3: Process Edge (2, 3)** Add 3 to `adj[2]`, Add 2 to `adj[3]` `adj[0] = [1, 2] adj[1] = [0] adj[2] = [0, 3] adj[3] = [2]`

1 UNDIRECTED EDGE LIST → ADJACENCY LIST

```
List<Integer>[] adj = new ArrayList[n];
for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
for (int[] edge : edges) {
    int u = edge[0], v = edge[1];
    adj[u].add(v); // Forward link u → v
    adj[v].add(u); // Reverse link v → u (Undirected!)
}
```

2 DIRECTED EDGE LIST → ADJACENCY LIST

```
List<Integer>[] adj = new ArrayList[n];
for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
for (int[] edge : edges) {
    int u = edge[0], v = edge[1];
    adj[u].add(v); // ONE-WAY link u → v only!
}
```

🔑 KEY DIFFERENCE: Undirected graphs require TWO list insertions (`adj[u].add(v)` AND `adj[v].add(u)`), while Directed graphs require ONLY ONE (`adj[u].add(v)`)!

💡 AHA MOMENT #3: WEIGHTED REPRESENTATIONS & GRID NEIGHBORS

For Weighted Graphs `[u, v, weight]`, represent neighbors as `int[] {neighborNode, weight}` inside `List[]`.
For Grids `grid[r][c]`, generate neighbors dynamically using Direction Arrays `DIRS\_4` (4-way) or `DIRS\_8` (8-way including diagonals)!

1 WEIGHTED GRAPH ('LIST []')

```
List<int[]>[] adj = new ArrayList[n];
for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
for (int[] edge : edges) { // [u, v, weight]
    int u = edge[0], v = edge[1], w = edge[2];
    adj[u].add(new int[]{v, w});
    adj[v].add(new int[]{u, w}); // Undirected
}
```

2 GRID 4-DIR VS 8-DIR OFFSETS

```
// 4-Directional Offsets (Up, Down, Left, Right)
int[][] DIRS_4 = {{0,1},{1,0},{0,-1},{-1,0}};

// 8-Directional Offsets (Includes Diagonals!)
int[][] DIRS_8 = {
    {-1,-1},{-1,0},{-1,1},
    { 0,-1},           { 0, 1},
    { 1,-1},{ 1, 0},{ 1, 1}
};
```

🌐 COORDINATE MAPPING / FLATTENING 2D GRID → 1D NODE ID

FLATTENING 2D (r, c) TO 1D UNIQUE NODE ID: `id = (row * TOTAL_COLUMNS) + col;` **RECONSTRUCTING 2D (r, c) FROM 1D NODE ID:** `row = id / TOTAL_COLUMNS;` `col = id % TOTAL_COLUMNS;` Example (Grid 3×4, cell (1, 2)): `id = 1 * 4 + 2 = 6!` `row = 6 / 4 = 1,` `col = 6 % 4 = 2!`

🚀 KEY TAKEAWAY: Coordinate flattening maps 2D grid cells directly into Disjoint Set Union (DSU) parent arrays!

 INPUT PATTERN TO GRAPH TEMPLATE RECOGNITION FLOWCHART

 PRINTABLE GRAPH BUILDER TEMPLATE LIBRARY

Input Format	Target Data Structure	Core Builder Loop
<code>edges[u, v]</code>	Undirected <code>List<Integer>[]</code>	<code>adj[u].add(v); adj[v].add(u);</code>
<code>prerequisites[a, b]</code>	Directed <code>List<Integer>[]</code>	<code>adj[b].add(a); indegree[a]++;</code>
<code>times[u, v, w]</code>	Weighted <code>List<int[]>[]</code>	<code>adj[u].add(new int[]{v, w});</code>
<code>grid[r][c]</code>	Implicit 2D Matrix Offsets	<code>for (int[] d : DIRS_4) nr = r + d[0];</code>

 **FAANG COACH TIP:** Pick your graph construction template within 5 seconds based on input parameter names!

**💡 AHA MOMENT #4: MULTI-SOURCE BFS PARALLEL EXPANSION**

Standard BFS starts from 1 source node. ****Multi-Source BFS**** enqueues ALL initial starting sources simultaneously into the Queue BEFORE starting the loop! This simulates ****parallel propagation**** (e.g. fire spreading, rotting oranges, water expansion) and computes shortest distance to ANY source in $O(V + E)$!

1 RECURSIVE DFS TRAVERSAL TEMPLATE

```
private void dfs(int u, List<Integer>[] adj, boolean[] visited) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) dfs(v, adj, visited);
    }
}
```

2 BFS LEVEL TRAVERSAL TEMPLATE

```
Queue<Integer> q = new ArrayDeque<>();
q.offer(start); visited[start] = true;
while (!q.isEmpty()) {
    int u = q.poll();
    for (int v : adj[u]) {
        if (!visited[v]) { visited[v] = true; q.offer(v); }
    }
}
```

🧠 VISUALIZING MULTI-SOURCE BFS QUEUE EXPANSION (ROTTING ORANGES)

Initial Grid: [2, 0, 1] Enqueue ALL rotten oranges (2) at minute 0: [1, 1, 0] Queue: [(0,0), (0,2)] [0, 1, 2] Minute 1: Poll (0,0) & (0,2) → Rot adjacent fresh oranges → Queue: [(1,0), (1,2)] Minute 2: Poll (1,0) & (1,2) → Rot adjacent (1,1) → Queue: [(1,1)] Total Minutes = 2!

🚩 **KEY TAKEAWAY:** Set `visited[neighbor] = true` IMMEDIATELY upon pushing into Queue to prevent duplicate node enqueueing!

**💡 AHA MOMENT #5: KAHN'S IN-DEGREE BFS ALGORITHM**

Topological Sort orders vertices in a DAG such that for every edge $u \rightarrow v$, node u appears before v .

1. Compute `indegree[i]` (number of incoming prerequisites) for all nodes.
2. Enqueue all nodes with `indegree == 0` (courses with zero prerequisites).
3. Poll node u , add to topological order, decrement `indegree[v]` for all neighbors. If `indegree[v] == 0`, enqueue v !
4. If processed count $< V$, a **CYCLE exists**!

🔧 KAHN'S TOPOLOGICAL SORT BFS TEMPLATE

```
public int[] findOrder(int numCourses, int[][] prerequisites) {
    List<Integer>[] adj = new ArrayList[numCourses];
    int[] indegree = new int[numCourses];
    for (int i = 0; i < numCourses; i++) adj[i] = new ArrayList<>();
    for (int[] p : prerequisites) {
        adj[p[1]].add(p[0]); // p[1] → p[0]
        indegree[p[0]]++;
    }
    Queue<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < numCourses; i++) if (indegree[i] == 0) q.offer(i);
    int[] order = new int[numCourses];
    int idx = 0;
    while (!q.isEmpty()) {
        int u = q.poll();
        order[idx++] = u;
        for (int v : adj[u]) {
            if (--indegree[v] == 0) q.offer(v); // Prerequisite satisfied!
        }
    }
    return idx == numCourses ? order : new int[0]; // Empty array if cycle detected!
}
```

⚠️ **CYCLE DETECTION:** If `idx != numCourses`, it means some nodes remained stuck with `indegree > 0`, proving the graph contains a cycle!

💡 AHA MOMENT #6: DSU PATH COMPRESSION & UNION BY RANK

DSU tracks connected components in dynamic graphs with near-constant $O(\alpha(N)) \approx O(1)$ time per operation!

- **Path Compression:** Point visited nodes directly to the root during `find(i)`: `parent[i] = find(parent[i])`.
- **Union by Rank:** Attach smaller tree under larger tree root to keep tree height minimal!

🔧 STANDARD FAANG DSU CLASS IMPLEMENTATION

```
class UnionFind {
    private int[] parent;
    private int[] rank;
    public int count; // Number of connected components

    public UnionFind(int n) {
        parent = new int[n]; rank = new int[n]; count = n;
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    public int find(int i) {
        if (parent[i] == i) return i;
        return parent[i] = find(parent[i]); // Path Compression!
    }

    public boolean union(int i, int j) {
        int rootI = find(i), rootJ = find(j);
        if (rootI != rootJ) {
            if (rank[rootI] < rank[rootJ]) parent[rootI] = rootJ;
            else if (rank[rootI] > rank[rootJ]) parent[rootJ] = rootI;
            else { parent[rootJ] = rootI; rank[rootI]++; }
            count--; return true; // Successfully merged!
        }
        return false; // Already connected → Cycle detected!
    }
}
```

🚩 **KEY TAKEAWAY:** If `union(u, v)` returns `false` (meaning `find(u) == find(v)`), edge (u, v) is a **Redundant Connection** creating a cycle!

1 DIJKSTRA'S SHORTEST PATH ($O((V+E) \log V)$)

```
public int[] dijkstra(int n, List<int[]>[] adj, int src) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) →
Integer.compare(a[1], b[1]));
    pq.offer(new int[]{src, 0});
    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int u = curr[0], d = curr[1];
        if (d > dist[u]) continue;
        for (int[] edge : adj[u]) {
            int v = edge[0], w = edge[1];
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.offer(new int[]{v, dist[v]});
            }
        }
    }
    return dist;
}
```

2 KRUSKAL'S MST ($O(E \log E)$)

```
public int minCostConnectPoints(int[][] edges, int n) {
    Arrays.sort(edges, (a, b) → Integer.compare(a[2], b[2]));
    UnionFind dsu = new UnionFind(n);
    int mstCost = 0, edgesAdded = 0;
    for (int[] e : edges) {
        if (dsu.union(e[0], e[1])) {
            mstCost += e[2];
            if (++edgesAdded == n - 1) break;
        }
    }
    return mstCost;
}
```

🚩 **ALGORITHM SELECTION:** Unweighted Shortest Path → ****BFS****. Non-Negative Weighted Shortest Path → ****Dijkstra****. Minimum Spanning Tree → ****Kruskal / Prim****.

 GRAPH INTERVIEW DECISION TREE (MERMAID)

graph TD
 A[What is the Graph Problem Type?] --> B[Grid Island / Flood Fill / Rotting Oranges?]
 A --> C[Course Prerequisites / Topological Order?]
 A --> D[Weighted Shortest Path?]
 A --> E[Minimum Cost to Connect All Nodes?]
 A --> F[Dynamic Connectivity / Cycle Check?]
 B --> B1[Grid DFS / Multi-Source BFS]
 C --> C1[Kahn's BFS In-Degree Topological Sort]
 D --> D1[Dijkstra PriorityQueue]
 E --> E1[Kruskal's MST + DSU]
 F --> F1[Disjoint Set Union DSU]

 PRINTABLE GRAPH ALGORITHMS CHEAT SHEET

Algorithm	Target Problem	Data Structure	Time Complexity
Grid DFS / BFS	Number of Islands (LC 200)	<code>visited[][]</code> + Matrix Offsets	$O(M \cdot N)$
Multi-Source BFS	Rotting Oranges (LC 994)	Queue (Enqueue all sources)	$O(M \cdot N)$
Kahn's TopoSort	Course Schedule (LC 207/210)	<code>indegree[]</code> + Queue	$O(V + E)$
Disjoint Set Union	Redundant Connection (LC 684)	<code>parent[]</code> + <code>rank[]</code>	$O(E \cdot \alpha(V))$
Dijkstra Shortest Path	Network Delay Time (LC 743)	Min PriorityQueue <code>[node, dist]</code>	$O((V + E) \log V)$
Kruskal's MST	Min Cost Connect (LC 1584)	Edge Array + DSU	$O(E \log E)$

 **FAANG COACH TIP:** Print this page and review it 10 minutes before your technical interview!



1. UNDIRECTED GRAPH BUILDER

```
List<Integer>[] buildGraph(int n, int[][] edges) {
    List<Integer>[] adj = new ArrayList[n];
    for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
    for (int[] e : edges) {
        adj[e[0]].add(e[1]); adj[e[1]].add(e[0]);
    }
    return adj;
}
```

2. DIRECTED GRAPH BUILDER

```
List<Integer>[] buildDirected(int n, int[][] edges) {
    List<Integer>[] adj = new ArrayList[n];
    for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
    for (int[] e : edges) { adj[e[0]].add(e[1]); }
    return adj;
}
```

3. STANDARD BFS LEVEL TEMPLATE

```
Queue<Integer> q = new ArrayDeque<>();
q.offer(start); vis[start] = true;
while (!q.isEmpty()) {
    int sz = q.size();
    for (int i = 0; i < sz; i++) {
        int u = q.poll();
        for (int v : adj[u]) {
            if (!vis[v]) { vis[v] = true; q.offer(v); }
        }
    }
}
```

4. MULTI-SOURCE BFS TEMPLATE

```
Queue<int[]> q = new ArrayDeque<>();
// Enqueue all initial sources first!
for (int[] src : sources) { q.offer(src); vis[src[0]][src[1]] = true; }
while (!q.isEmpty()) {
    int[] curr = q.poll();
    for (int[] d : DIRS_4) {
        int nr = curr[0] + d[0], nc = curr[1] + d[1];
        if (valid(nr, nc) && !vis[nr][nc]) { vis[nr][nc] = true;
        q.offer(new int[]{nr, nc}); }
    }
}
```

🚀 **REUSABLE CODE:** Copy-paste these verified templates directly into your interview whiteboard or IDE!

5. TOPOLOGICAL SORT (KAHN'S BFS)

```
int[] topoSort(int n, int[][] edges) {
    List<Integer>[] adj = buildDirected(n, edges);
    int[] in = new int[n];
    for (int[] e : edges) in[e[1]]++;
    Queue<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < n; i++) if (in[i] == 0) q.offer(i);
    int[] res = new int[n]; int idx = 0;
    while (!q.isEmpty()) {
        int u = q.poll(); res[idx++] = u;
        for (int v : adj[u]) if (--in[v] == 0) q.offer(v);
    }
    return idx == n ? res : new int[0];
}
```

6. DIJKSTRA SHORTEST PATH

```
int[] dijkstra(int n, List<int[]>[] adj, int src) {
    int[] dist = new int[n]; Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[1] - b[1]);
    pq.offer(new int[]{src, 0});
    while (!pq.isEmpty()) {
        int[] curr = pq.poll(); int u = curr[0], d = curr[1];
        if (d > dist[u]) continue;
        for (int[] e : adj[u]) {
            if (dist[u] + e[1] < dist[e[0]]) {
                dist[e[0]] = dist[u] + e[1]; pq.offer(new int[]{e[0],
                    dist[e[0]]});
            }
        }
    }
    return dist;
}
```

🔑 **KEY TAKEAWAY:** Memorize these 6 core templates and you can solve 95% of FAANG Graph problems!

1. LEETCODE 733 — FLOOD FILL

 **Easy ★★★★★** Grid DFS Color Replacement Amazon

Problem:

Perform flood fill on 2D image starting at `(sr, sc)` replacing matching color with `color`.

```
public int[][] floodFill(int[][] image, int sr, int sc, int color) {
    int origColor = image[sr][sc];
    if (origColor != color) dfs(image, sr, sc, origColor, color);
    return image;
}

private void dfs(int[][] img, int r, int c, int orig, int color) {
    if (r < 0 || r ≥ img.length || c < 0 || c ≥ img[0].length ||
    img[r][c] != orig) return;
    img[r][c] = color; // Change color (acts as visited!)
    dfs(img, r + 1, c, orig, color); dfs(img, r - 1, c, orig, color);
    dfs(img, r, c + 1, orig, color); dfs(img, r, c - 1, orig, color);
}
```

2. LEETCODE 1971 — FIND IF PATH EXISTS IN GRAPH

 **Easy ★★★★★** BFS / DSU Connectivity Amazon

Problem:

Determine if valid path exists between `source` and `destination` in undirected graph.

```
public boolean validPath(int n, int[][] edges, int source, int
destination) {
    UnionFind dsu = new UnionFind(n);
    for (int[] e : edges) dsu.union(e[0], e[1]);
    return dsu.find(source) == dsu.find(destination);
}
```

 **AHA MOMENT (LC 733):** Mutating `image[r][c] = color` directly in-place serves as the `visited` check, saving auxiliary memory!

3. LEETCODE 200 — NUMBER OF ISLANDS

🔥 Medium ★★★★★ Grid DFS Island Sink [Amazon](#) [Meta](#) [Google](#)

Problem:

Count total number of 4-directionally connected land islands ('1').

```
public int numIslands(char[][] grid) {
    int count = 0;
    for (int r = 0; r < grid.length; r++) {
        for (int c = 0; c < grid[0].length; c++) {
            if (grid[r][c] == '1') {
                count++; dfs(grid, r, c); // Sink island!
            }
        }
    }
    return count;
}

private void dfs(char[][] grid, int r, int c) {
    if (r < 0 || r ≥ grid.length || c < 0 || c ≥ grid[0].length ||
        grid[r][c] ≠ '1') return;
    grid[r][c] = '0'; // Sink land to '0'
    dfs(grid, r + 1, c); dfs(grid, r - 1, c);
    dfs(grid, r, c + 1); dfs(grid, r, c - 1);
}
```

4. LEETCODE 695 — MAX AREA OF ISLAND

🔥 Medium ★★★★★ Grid DFS Area Accumulator [Amazon](#) [Meta](#)

Problem:

Return maximum area (number of connected 1s) of an island in grid.

```
public int maxAreaOfIsland(int[][] grid) {
    int maxArea = 0;
    for (int r = 0; r < grid.length; r++) {
        for (int c = 0; c < grid[0].length; c++) {
            if (grid[r][c] == 1) maxArea = Math.max(maxArea, dfs(grid,
                r, c));
        }
    }
    return maxArea;
}

private int dfs(int[][] grid, int r, int c) {
    if (r < 0 || r ≥ grid.length || c < 0 || c ≥ grid[0].length ||
        grid[r][c] ≠ 1) return 0;
    grid[r][c] = 0; // Sink land
    return 1 + dfs(grid, r + 1, c) + dfs(grid, r - 1, c) + dfs(grid, r,
        c + 1) + dfs(grid, r, c - 1);
}
```

💡 **AHA MOMENT (LC 200):** Sinking land from '1' to '0' during DFS eliminates the need for an extra 'visited[][]' boolean array!

5. LEETCODE 994 — ROTTING ORANGES

Medium ★★★★★ Multi-Source BFS [Amazon](#) [Google](#)

Problem:

Minimum minutes to rot all fresh oranges using Multi-Source BFS.

```
public int orangesRotting(int[][] grid) {
    int rows = grid.length, cols = grid[0].length, fresh = 0, minutes = 0;
    Queue<int[]> q = new ArrayDeque<>();
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == 2) q.offer(new int[]{r, c});
            else if (grid[r][c] == 1) fresh++;
        }
    }
    if (fresh == 0) return 0;
    int[][] DIRS = {{0,1},{1,0},{0,-1},{-1,0}};
    while (!q.isEmpty() && fresh > 0) {
        int size = q.size(); minutes++;
        for (int i = 0; i < size; i++) {
            int[] curr = q.poll();
            for (int[] d : DIRS) {
                int nr = curr[0] + d[0], nc = curr[1] + d[1];
                if (nr ≥ 0 && nr < rows && nc ≥ 0 && nc < cols &&
                    grid[nr][nc] == 1) {
                    grid[nr][nc] = 2; fresh--; q.offer(new int[]{nr, nc});
                }
            }
        }
    }
    return fresh == 0 ? minutes : -1;
}
```

6. LEETCODE 133 — CLONE GRAPH

Medium ★★★★★ DFS HashMap Node Mapping [Amazon](#) [Meta](#)

Problem:

Deep copy a connected undirected graph.

```
private Map<Node, Node> visited = new HashMap<>();
public Node cloneGraph(Node node) {
    if (node == null) return null;
    if (visited.containsKey(node)) return visited.get(node);
    Node clone = new Node(node.val);
    visited.put(node, clone);
    for (Node neighbor : node.neighbors) {
        clone.neighbors.add(cloneGraph(neighbor));
    }
    return clone;
}
```

 **AHA MOMENT (LC 133):** `HashMap` tracks original-to-clone node mapping to prevent infinite recursion on graph cycles!



7. LEETCODE 207 — COURSE SCHEDULE I

Medium ★★★★★ Kahn's BFS In-Degree Cycle Check [Amazon](#) [Meta](#) [Google](#)**Problem:**

Determine if it is possible to finish all 'numCourses' given prerequisites.

```
public boolean canFinish(int numCourses, int[][] prerequisites) {
    List<Integer>[] adj = new ArrayList[numCourses];
    int[] indegree = new int[numCourses];
    for (int i = 0; i < numCourses; i++) adj[i] = new ArrayList<>();
    for (int[] p : prerequisites) {
        adj[p[1]].add(p[0]); indegree[p[0]]++;
    }
    Queue<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < numCourses; i++) if (indegree[i] == 0)
        q.offer(i);
    int count = 0;
    while (!q.isEmpty()) {
        int u = q.poll(); count++;
        for (int v : adj[u]) if (--indegree[v] == 0) q.offer(v);
    }
    return count == numCourses;
}
```

8. LEETCODE 210 — COURSE SCHEDULE II

Medium ★★★★★ Topological Ordering Array [Amazon](#) [Google](#)**Problem:**

Return valid ordering of courses to finish all requirements.

```
public int[] findOrder(int numCourses, int[][] prerequisites) {
    List<Integer>[] adj = new ArrayList[numCourses];
    int[] indegree = new int[numCourses];
    for (int i = 0; i < numCourses; i++) adj[i] = new ArrayList<>();
    for (int[] p : prerequisites) {
        adj[p[1]].add(p[0]); indegree[p[0]]++;
    }
    Queue<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < numCourses; i++) if (indegree[i] == 0)
        q.offer(i);
    int[] res = new int[numCourses]; int idx = 0;
    while (!q.isEmpty()) {
        int u = q.poll(); res[idx++] = u;
        for (int v : adj[u]) if (--indegree[v] == 0) q.offer(v);
    }
    return idx == numCourses ? res : new int[0];
}
```

🧠 **MEMORY BOX:** Prerequisites '[a, b]' means course 'b' **MUST** be taken before 'a', establishing directed edge 'b → a'!



9. LEETCODE 684 — REDUNDANT CONNECTION

Medium ★★★★★ DSU Cycle Edge Detection Amazon

Problem:

Return edge that can be removed so tree has zero cycles.

```
public int[] findRedundantConnection(int[][] edges) {
    int n = edges.length;
    UnionFind dsu = new UnionFind(n + 1);
    for (int[] edge : edges) {
        if (!dsu.union(edge[0], edge[1])) return edge; // Cycle edge!
    }
    return new int[0];
}
```

10. LEETCODE 743 — NETWORK DELAY TIME

Medium ★★★★★ Dijkstra Shortest Path Amazon Google

Problem:

Minimum time for all N nodes to receive signal from source K .

```
public int networkDelayTime(int[][] times, int n, int k) {
    List<int[]>[] adj = new ArrayList[n + 1];
    for (int i = 1; i ≤ n; i++) adj[i] = new ArrayList<>();
    for (int[] t : times) adj[t[0]].add(new int[]{t[1], t[2]});
    int[] dist = new int[n + 1]; Arrays.fill(dist, Integer.MAX_VALUE);
    dist[k] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) →
Integer.compare(a[1], b[1]));
    pq.offer(new int[]{k, 0});
    while (!pq.isEmpty()) {
        int[] curr = pq.poll(); int u = curr[0], d = curr[1];
        if (d > dist[u]) continue;
        for (int[] edge : adj[u]) {
            int v = edge[0], w = edge[1];
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w; pq.offer(new int[]{v, dist[v]});
            }
        }
    }
    int maxDist = 0;
    for (int i = 1; i ≤ n; i++) {
        if (dist[i] == Integer.MAX_VALUE) return -1;
        maxDist = Math.max(maxDist, dist[i]);
    }
    return maxDist;
}
```

💡 **AHA MOMENT (LC 684):** The FIRST edge where `dsu.union(u, v)` returns `false` is the exact edge creating the cycle!

11. LEETCODE 269 — ALIEN DICTIONARY

Hard ★★★★★
Char Graph + TopoSort
Amazon
Meta
Google

Problem:
Determine character ordering from sorted alien dictionary words.

```

public String alienOrder(String[] words) {
    Map<Character, Set<Character>> adj = new HashMap<>();
    Map<Character, Integer> indegree = new HashMap<>();
    for (String w : words) for (char c : w.toCharArray()) {
        adj.putIfAbsent(c, new HashSet<>()); indegree.putIfAbsent(c, 0);
    }
    for (int i = 0; i < words.length - 1; i++) {
        String w1 = words[i], w2 = words[i+1];
        if (w1.length() > w2.length() && w1.startsWith(w2)) return "";
        for (int j = 0; j < Math.min(w1.length(), w2.length()); j++) {
            char c1 = w1.charAt(j), c2 = w2.charAt(j);
            if (c1 != c2) {
                if (!adj.get(c1).contains(c2)) {
                    adj.get(c1).add(c2); indegree.put(c2,
indegree.get(c2) + 1);
                }
                break;
            }
        }
    }
    Queue<Character> q = new ArrayDeque<>();
    for (char c : indegree.keySet()) if (indegree.get(c) == 0)
q.offer(c);
    StringBuilder sb = new StringBuilder();
    while (!q.isEmpty()) {
        char u = q.poll(); sb.append(u);
        for (char v : adj.get(u)) {
            indegree.put(v, indegree.get(v) - 1);
            if (indegree.get(v) == 0) q.offer(v);
        }
    }
    return sb.length() == indegree.size() ? sb.toString() : "";
}

```

★ THE 4 GOLDEN LAWS OF GRAPHS

- 1. **Always Track Visited:** Graphs contain cycles! Mark `visited[u] = true` immediately upon discovery.
- 2. **Unweighted Shortest Path → BFS:** BFS guarantees shortest path level-by-level in unweighted graphs.
- 3. **Dependencies → Topological Sort (Kahn BFS):** Enqueue nodes with `indegree == 0`.
- 4. **Dynamic Connectivity & Cycle Check → DSU:** Path compression gives $O(\alpha(N))$ union/find ops.

👉 Next Master Topic: Topic 13 → Dynamic Programming Masterclass

🚩 TOPIC 12 GRAPHS MASTERCLASS COMPLETE! Turn the page to Topic 13: Dynamic Programming Masterclass!